

CS 5326: Advanced Generative AI and Agents

PA 1: The Modern Transformer LM

Design and Lead: Mohammad Ali Wajid

1 Assignment Overview

In this assignment, you will build all the components needed to train a modern Transformer language model (LM) from scratch and train a model.

What you will implement

1. `Transformer language model (LM)`
2. The `cross-entropy loss function` and the `AdamW optimizer`
3. The `data loader`, `training loop`, and `text-generation procedure`

What you will run

1. `Transformer LM training on the TinyStories dataset`.
2. `Sample generation` and `evaluation of perplexity` using the trained Transformer LM.

What you will have built

By the end of this assignment, you will have implemented and trained a language model capable of generating coherent TinyStories-style English. The following sample was generated by the 19-million-parameter model used to calibrate this assignment:

One day, Mia found a shiny red box in the park. She was so excited! Mia started to open the box, and she found all kinds of colorful things. She picked out a shiny red toy car and a bright red ball.

Mia went to her friend Tom and showed him the toy car. Tom said, ``Wow, that toy is so cool! I wish I had a toy car like that.'' Mia thought for a moment and then said, ``Maybe next time you will learn to be less greedy, Tom.''

Mia and Tom played with the shiny red car all day long. The red car became better and better than before.

Your exact generations will vary with training and decoding choices, but coherent short stories are an achievable outcome.

How the assignment is graded

The programming implementation is worth 70% of the assignment grade and is evaluated with unit tests. The repository includes public tests that you can run while developing. Grading may also use hidden tests that exercise the same implementation contracts described in this manual through the same adapter interface. The remaining 30% is based on `REPORT.md`, including the training evidence, hyperparameter study, final evaluation, generated samples, and analysis requested in [Section 7.3](#). We do not disclose a public-to-hidden breakdown within the unit-test portion.

What you can use

We expect you to build each component from scratch. In particular, you may not use any definitions from `torch.nn`, `torch.nn.functional`, or `torch.optim` except for the following:

- `torch.nn.Parameter`
- [Container classes in torch.nn](#) (e.g., `Module`, `ModuleList`, `Sequential`, etc.).
- Initialization utilities in `torch.nn.init`, including `torch.nn.init.trunc_normal_`
- The `torch.optim.Optimizer` base class

Unless explicitly forbidden by a problem, you may use functions from the top-level `torch` namespace and ordinary tensor methods. You may not use implementations from `torch.nn`, `torch.nn.functional`, or `torch.optim` beyond the exceptions listed above. In addition, if a function is itself an assignment deliverable, you may not call an equivalent built-in implementation to perform that function for you.

What the code looks like

The assignment code, public tests, and this writeup are provided in the course assignment repository. [Section 2](#) explains how to clone the repository, create the environment, run the tests, and download the dataset.

Where to get the dataset

This assignment uses TinyStories [\[R. Eldan et al., 2023\]](#). This assignment provides a fixed tokenizer and pre-tokenized train and validation streams. Instructions for downloading them appear in [Section 2.4](#). You will not train or implement a tokenizer.

How to submit

From the repository root, run:

```
bash make_submission.sh
```

The supplied script creates `submission.zip`. Upload that single archive to the LMS. It must contain your implementation, completed `tests/adapters.py`, `REPORT.md`, report figures, requested logs, and one model-only artifact named `final_model.pt`. The script excludes the downloaded dataset, Git metadata, virtual environments, caches, and all other model or optimizer checkpoints. Run the complete public test suite before packaging. Section 7.4 defines the exact model-artifact format and submission contents.

2 Code Repository, Environment, and Dataset

2.1 Clone the assignment repository

Clone the [PA1 repository](#), then enter the repository directory:

```
git clone https://github.com/alooboii/cs5326-pa1.git
cd cs5326-pa1
```

The repository contains this manual, public tests, `REPORT.md`, `make_submission.sh`, and a `src/` directory for your implementation. Write your implementation in `src/`; you do not need to create a separate top-level package or implementation directory. You may divide the implementation across files such as `layers.py`, `attention.py`, `model.py`, `optim.py`, and `train.py` inside `src/`.

The files under `tests/` define the interface used for grading:

- `tests/adapters.py` is the only file in the test harness that you should edit. Each adapter imports your implementation, constructs the relevant object, loads supplied weights when required, invokes your code, and returns its result.
- `tests/test_*.py` contains the public tests. Do not edit these files.

Adapters are **glue code only**. Do not implement linear algebra, attention, optimization, or other assignment

logic inside `tests/adapters.py`. You may choose your own package name, files, class names, and internal interfaces.

2.2 Environment

We manage the assignment environment with `uv` to make dependency installation reproducible and portable. Install `uv` using the [official installation instructions](#). You may instead use `pip install uv` or `brew install uv`. We recommend reading the short [uv guide to managing project dependencies](#). You will not regret it!

From the repository root, install the locked assignment environment with:

```
uv sync --frozen
```

You can then run any Python file you create inside that environment without manually activating a virtual environment:

```
uv run python <python_file_path>
```

2.3 Run the unit tests

Run the complete public test suite from the repository root:

```
uv run pytest
```

Initially, the implementation tests should fail with `NotImplementedError`. To connect your implementation to the tests, complete the corresponding functions in `tests/adapters.py`. Always invoke `pytest` through `uv` run from the repository root so it uses the locked assignment environment and resolves `tests/adapters.py` consistently.

Pytest accepts a test path after the command. With no path it discovers and runs the complete public suite. A file path runs that file, while a node ID of the form `path::test_name` runs one exact test:

```
# Run every public test.
uv run pytest

# Run one test file.
uv run pytest tests/test_layers.py

# Run one exact test function.
uv run pytest \
    tests/test_layers.py::test_embedding_lookup_and_repeated_indices
```

The `-k` option selects tests whose collected names match a keyword expression. It performs name selection rather than searching the test source code. Expressions may use `and`, `or`, and `not`; quote compound expressions so the shell passes them as one argument:

```
# Run tests with "linear" in their collected name.
uv run pytest -k linear

# Combine name filters.
uv run pytest -k "rope or attention"
uv run pytest -k "attention and not gqa"

# Combine a file path with a name filter.
uv run pytest tests/test_optim.py -k adamw
```

The following options are useful while developing:

Option	Behavior
<code>-v</code>	Show every collected test and its result; use <code>-vv</code> for still more detail.
<code>-q</code>	Reduce routine output. This is convenient for a final all-tests check.
<code>-x</code>	Stop immediately after the first failure.
<code>--maxfail=N</code>	Stop after exactly N failures.
<code>-s</code>	Disable output capture so that <code>print</code> output appears immediately.
<code>--lf</code>	Run only tests that failed in the preceding pytest invocation.
<code>--ff</code>	Run previous failures first, followed by the remaining selected tests.
<code>--collect-only</code>	List selected tests without executing them.

Options and selectors compose. For example, `uv run pytest tests/test_attention.py -k gqa -vv -x` runs only GQA-named tests from the attention file, displays their full node IDs, and stops after the first failure. A zero process exit code means every selected test passed; a nonzero exit code means at least one selected test failed or pytest encountered a collection or execution error. Run `uv run pytest -h` for the complete command-line reference.

Use focused commands for iteration, but run `uv run pytest` without a path or `-k` filter before submission. Each problem box in this manual names the adapter function or functions to complete after implementing that component in your own package.

2.4 Download the dataset

This assignment uses TinyStories [R. Eldan et al., 2023]. The course provides the public Hugging Face dataset `alooboi/pa1-tinystories`, containing fixed pretokenized versions of the complete TinyStories training and validation splits.

Download the tokenizer and the complete training and validation streams:

```
uv run hf download alooboi/pa1-tinystories \
  metadata.json tokenizer/tokenizer.json \
  data/train.bin data/validation.bin \
  --repo-type dataset \
  --local-dir data/tinystories
```

The resulting directory contains:

```
data/tinystories/
|-- metadata.json
|-- tokenizer/tokenizer.json
|-- data/train.bin
`-- data/validation.bin
```

The complete training stream contains 466,876,982 tokens from 2,119,719 stories. The validation stream contains 4,692,376 tokens from 21,990 stories. The public tests use small synthetic arrays and do not access these files. While bringing up your training loop, repeatedly optimizing one fixed minibatch from the training stream is a more useful end-to-end correctness check than beginning a long run immediately.

2.5 Using Kaggle or Google Colab

You may complete the assignment locally or use a free cloud GPU. On either Kaggle or Google Colab, first enable a GPU runtime and internet access, then install `uv`, clone or copy your repository into a writable directory, run `uv sync --frozen`, and download the public dataset with the command in [Section 2.4](#). The dataset itself is public and does not require a Hugging Face access token.

Kaggle mounts attached input datasets read-only under `/kaggle/input`. If you attach a snapshot of your repository as a Kaggle input, copy it into a directory under `/kaggle/working` before running or modifying it. A public Git repository may instead be cloned directly. For a private repository, use Kaggle’s secret storage

or upload a private repository snapshot; never paste a GitHub token into a notebook cell, source file, output log, or Git history. Write checkpoints and final artifacts under `/kaggle/working`, and save a notebook version or download those files before the session ends.

Colab's `/content` filesystem is also temporary. Clone the repository into `/content` and mount Google Drive for checkpoints, logs, and other work that must survive a disconnected runtime. It is usually faster to keep the downloaded token streams on the runtime's local disk and preserve only the smaller checkpoints and experimental artifacts in Drive. If the repository is private, retrieve credentials from Colab's secret storage and do not print them.

Cloud runtime limits and accelerator availability may change. Design the training script to save the resumable checkpoints from [Section 5.2](#); after reconnecting, restore the checkpoint and continue from its saved `next_step`. At the end, run `bash make_submission.sh` in the writable repository directory and download the resulting `submission.zip` for LMS upload.

2.6 Fixed tokenizer

The tokenizer is a fixed 8,192-token byte-level BPE [\[R. Sennrich et al., 2016\]](#); [\[C. Wang et al., 2019\]](#). Its vocabulary contains `<|endoftext|>`, abbreviated EOT. The embedding table and output head are *not* pre-trained: they are ordinary parameters that your model initializes and learns.

Byte-level BPE represents arbitrary UTF-8 text without an unknown-token problem, while frequent byte sequences are merged into larger tokens to shorten sequences. Tokenizer training and merge implementation are outside of the scope of this assignment. Load the provided `.json` file only when encoding prompts or decoding generated IDs:

```
from tokenizers import Tokenizer

tokenizer = Tokenizer.from_file(
    "data/tinystories/tokenizer/tokenizer.json"
)
prompt_ids = tokenizer.encode(
    "Once upon a time", add_special_tokens=False
).ids
text = tokenizer.decode(generated_ids, skip_special_tokens=False)
```

The pretokenized training stream already contains IDs, so do not decode and re-encode it during training. [Section 5.1](#) explains how to open these streams and sample next-token batches.

3 Transformer Language Model Architecture

A language model takes a batched sequence of integer token IDs (i.e., a `torch.Tensor` of shape `(batch_size, sequence_length)`) and returns next-token logits of shape `(batch_size, sequence_length, vocab_size)`. Applying softmax to the logits produces, at every input position, a probability distribution over the next token. During training, we use these predictions to calculate the cross-entropy loss between the predicted and actual next tokens. During inference, we use the distribution at the final sequence position to choose or sample the next token, append that token to the input sequence, and repeat.

In this part of the assignment, you will build this Transformer language model from scratch. We will begin with a high-level description of the model before progressively detailing the tensor conventions in [Section 3.2](#), primitive layers in [Section 3.3](#), the pre-norm block in [Section 3.4](#), and the integrated model in [Section 3.5](#).

3.1 Transformer LM

Given a sequence of token IDs, the Transformer language model uses a learned token embedding to map IDs to dense vectors, passes those vectors through `num_layers` Transformer blocks, applies a final RMSNorm, and

then uses a learned linear projection (the LM head) to produce next-token logits [A. Radford et al., 2018]. The model forward method returns logits, not probabilities; softmax is applied only where a loss or sampling distribution needs it.

Token Embeddings

The embedding layer maps an integer tensor of shape `(batch_size, sequence_length)` to a tensor of shape `(batch_size, sequence_length, d_model)`.

Pre-norm Transformer Block

After embedding, the activations pass through `num_layers` identically structured, independently parameterized Transformer blocks. Each block maps `(batch_size, sequence_length, d_model)` to the same shape, aggregates information across positions through causal self-attention, and transforms each position independently through a feed-forward network. We use the pre-norm arrangement, so each sub-layer receives an RMS-normalized input and its output is added back to the residual stream. A final RMSNorm is applied after the last block before the LM head.

The values below are fixed for the final TinyStories model. Your implementation should still accept these quantities as constructor arguments because the public tests use smaller configurations to check correctness.

Symbol	Variable	Meaning	Value
$ \mathcal{V} $	<code>vocab_size</code>	vocabulary size	8192
$n_{\max}$	<code>context_length</code>	maximum sequence length	256
d_{model}	<code>d_model</code>	residual-stream dimension	512
L	<code>num_layers</code>	number of Transformer blocks	4
h_q	<code>n_q_heads</code>	number of query heads	16
h_{kv}	<code>n_kv_heads</code>	number of key/value heads	4
$g = h_q/h_{kv}$	<code>group_size</code>	query heads per KV head	4
$d_h = d_{\text{model}}/h_q$	<code>head_dim</code>	attention-head dimension	32
d_{ff}	<code>d_ff</code>	SwiGLU hidden dimension	1344
Θ	<code>rope_theta</code>	RoPE frequency base	10,000
$\varepsilon_{\text{norm}}$	<code>norm_eps</code>	RMSNorm stability constant	10^{-5}

Throughout the architecture sections, i and j denote query/current and key/source token positions, respectively; a indexes an ordinary attention head; h indexes a key/value head (and therefore a GQA group); r indexes a query head within that group; and k indexes a RoPE coordinate pair/frequency. In the training sections, s denotes the global optimization step used by the learning-rate schedule.

The token embedding and LM head are separate, untied matrices, and every linear projection is bias-free. The fixed architecture does not use dropout anywhere in the embedding path, attention sublayer, SwiGLU sublayer, residual paths, or output path. Training choices such as microbatch size, gradient accumulation, learning rate, warmup, and evaluation frequency are hyperparameters; the values later in this manual are starting points that you may adjust or ablate.

Let B denote the runtime batch size and $n \leq n_{\max}$ the runtime sequence length. Given input tokens $x_1, \dots, x_n$, the logits at input position i parameterize the next-token conditional

$$p_{\theta}(x_{i+1} \mid x_{1:i}), \quad i \in \{1, \dots, n\}.$$

One forward pass produces logits $Z \in \mathbb{R}^{B \times n \times |\mathcal{V}|}$ for all of these conditionals in parallel. Causal masking ensures that the logits at position i use only tokens at positions at most i .

Figure 1 summarizes the complete architecture and the two pre-norm residual paths inside every block. Its final softmax denotes the conversion from logits to probabilities.

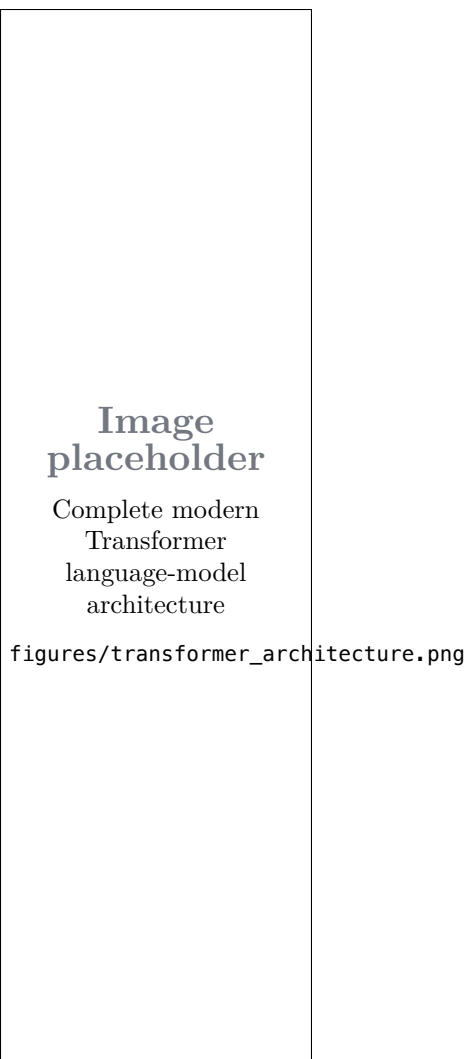


Figure 1: An overview of our Transformer language model.

In pseudocode, the required architecture is:

```
def forward(token_ids, token_positions=None):
    x = token_embedding(token_ids)

    for block in blocks:
        x = block(x, token_positions=token_positions)

    x = final_norm(x)
    logits = lm_head(x)
    return logits
```

The language model, every Transformer block, and the attention module therefore accept an optional token-position argument. `token_positions` must have final dimension `sequence_length`; any leading dimensions must be broadcastable to the corresponding batch-like dimensions of the input. Thus both `(sequence_length,)` and `(batch_size, sequence_length)` are valid in the ordinary batched case. When `token_positions` is **None**, attention uses the default positions `0, ..., sequence_length - 1`. When positions are supplied explicitly, the language model passes them unchanged to every block, and each block passes them unchanged to its attention sublayer.

Inside each block, the order is exactly:

```
def forward(x, token_positions=None):
    x = x + attention(
        attention_norm(x),
        token_positions=token_positions,
    )
    x = x + swiglu(ffn_norm(x))
    return x
```

The two normalization modules within each block have separate learned parameters, as does every Transformer block.

3.2 Batch-like dimensions and tensor notation

Throughout the Transformer, we will be performing the same computation applied to many batch-like inputs. Here are a few examples:

- **Elements of a batch:** we apply the same Transformer forward operation on each batch element.
- **Sequence length:** position-wise operations like RMSNorm and feed-forward operate identically on each position of a sequence.
- **Attention heads:** the attention operation is batched across attention heads in a multi-head attention operation.

It is useful to have an ergonomic way of performing such operations in a way that fully utilizes the GPU and is easy to read and understand. Many PyTorch operations can take excess batch-like dimensions at the start of a tensor and repeat or broadcast the operation across these dimensions efficiently.

Because of this, it is helpful to assume that your functions may be given additional batch-like dimensions and to keep those dimensions at the start of the PyTorch shape. To organize tensors so they can be batched in this manner, they might need to be shaped using many steps of `view`, `reshape`, and `transpose`. This can be hard to read.

A more ergonomic option is Einstein summation notation through `einops.einsum`. The two key operations are `einsum`, which can perform tensor contractions across named dimensions, and `rearrange`, which can reorder, concatenate, and split dimensions. Much of the tensor work in a Transformer is some combination of dimension rearrangement, tensor contraction, and a pointwise nonlinear function, so expressing the axes by name makes the implementation easier to read and less dependent on a particular number of leading batch dimensions.

We strongly recommend learning and using `einsum` notation for this assignment. Read the [einops basics documentation](#) before implementing the tensor rearrangements in attention. The package is already part of the assignment's uv environment, and you may use `einops.einsum` and `einops.rearrange` throughout your implementation.

`Einsum` notation can handle arbitrary leading batch dimensions, but its other important benefit is that the axis names document the intended input and output shapes. For tensors that are not naturally expressed through `einsum`, use tensor type hints from `jaxtyping`. Despite its name, `jaxtyping` is not specific to JAX and supports PyTorch tensors. An annotation such as `Float[Tensor, "batch sequence d_model"]` records the semantic meaning of each axis alongside the ordinary Python type. Together, named `einsum` axes, type hints, and short shape comments make tensor-shape mistakes substantially easier to locate.

Use named shape comments while developing attention:

```
# x: [batch_size, sequence_length, d_model]
# q: [batch_size, kv_head, query_in_group, sequence_length, head_dim]
# k, v: [batch_size, kv_head, sequence_length, head_dim]
# scores: [batch_size, kv_head, query_in_group, query, key]
```


`einops.einsum` often makes contractions easier to audit. Plain PyTorch reshape and transpose operations are also valid, but verify the axis ordering before flattening dimensions. `reshape` may copy when an input is not contiguous, while `view` requires a compatible memory layout.

3.2.1 Mathematical Notation and Memory Ordering

Many machine-learning implementations place feature vectors on the final tensor axis. With a learned matrix $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ stored in PyTorch convention and a row-vector input $x \in \mathbb{R}^{1 \times d_{\text{in}}}$, the corresponding code-level transformation is

$$y = xW^\top.$$

This arrangement batches naturally over any leading dimensions.

In conventional linear-algebra notation, we will mostly write vectors as columns, so the same transformation appears as

$$y = Wx.$$

Thus, mathematical expressions in this manual may use Wx , while PyTorch tensors whose feature dimension is last apply the stored matrix with a transpose. If you use `einsum`, this distinction is straightforward as long as the axes are labeled correctly. **This convention is important because tests inject weights with this exact orientation.**

3.3 Basic Building Blocks: Linear and Embedding Modules

3.3.1 Parameter Initialization

Training neural networks effectively often requires careful initialization of the model parameters—bad initializations can lead to undesirable behavior such as vanishing or exploding gradients. Pre-norm Transformers are unusually robust to initialization, but initialization can still have a significant impact on training speed and convergence. Since this assignment is already long, we will give you some approximate initializations that should work well for most cases. For now, use:

- **Linear weights:** $\mathcal{N}(\mu = 0, \sigma^2 = \frac{2}{d_{\text{in}} + d_{\text{out}}})$ truncated at $[-3\sigma, 3\sigma]$.
- **Embedding:** $\mathcal{N}(\mu = 0, \sigma^2 = 1)$ truncated at $[-3, 3]$.
- **RMSNorm:** $\mathbb{1}$.

You should use `torch.nn.init.trunc_normal_` to initialize the truncated normal weights.

3.3.2 Linear and Embedding Modules

Linear layers are a fundamental building block of Transformers and neural nets in general. First, you will implement your own `Linear` class that inherits from `torch.nn.Module` and performs a linear transformation:

$$y = xW^\top.$$

Note that we do not include a bias term, following most modern LMs.

Problem: Implement the linear module (3 points)

Deliverable: Implement a `Linear` class that inherits from `torch.nn.Module` and applies a bias-free linear transformation. Its interface should follow PyTorch's `nn.Linear`, except that it has no bias argument or bias parameter. We recommend the following interface:

```
def __init__(
    self,
```

```

    in_features: int,
    out_features: int,
    device: torch.device | None = None,
    dtype: torch.dtype | None = None,
):
    ...

def forward(self, x: torch.Tensor) -> torch.Tensor:
    ...

```

`in_features: int` Final dimension of the input.

`out_features: int` Final dimension of the output.

`device: torch.device | None` Device on which to store the parameter.

`dtype: torch.dtype | None` Data type of the parameter.

The forward method must accept an input of shape $(\dots, \text{in_features})$ and return an output of shape $(\dots, \text{out_features})$. Subclass `nn.Module`, call the superclass constructor, and store W —not W^T —as an `nn.Parameter` of shape $(\text{out_features}, \text{in_features})$. Do not use `nn.Linear` or `nn.functional.linear`. Initialize the weight using the prescription in [Section 3.3.1](#).

To test your implementation, complete `adapters.run_linear` and run `uv run pytest -k test_linear`. The adapter should instantiate your module, load the supplied weight matrix, and call the module; it must not implement the matrix multiplication itself.

As discussed above, the first layer of the Transformer is an embedding layer that maps integer token IDs into a vector space of dimension `d_model`. We will implement a custom `Embedding` class that inherits from `torch.nn.Module` (so you should not use `nn.Embedding`). The forward method should select the embedding vector for each token ID by indexing into an embedding matrix of shape $(\text{vocab_size}, \text{d_model})$ using a `torch.LongTensor` of token IDs with shape $(\text{batch_size}, \text{sequence_length})$.

An embedding stores $E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$. Integer indexing maps token IDs of shape $[B, n]$ to $[B, n, d_{\text{model}}]$. Directly indexing a parameter remains differentiable with respect to the selected rows.

Problem: Implement the embedding module (3 points)

Deliverable: Implement an `Embedding` class that inherits from `torch.nn.Module` and performs an embedding lookup. We recommend the following interface:

```

def __init__(
    self,
    num_embeddings: int,
    embedding_dim: int,
    device: torch.device | None = None,
    dtype: torch.dtype | None = None,
):
    ...

def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
    ...

```

`num_embeddings: int` Number of entries in the vocabulary.

`embedding_dim: int` Dimension of each embedding vector.

`device: torch.device | None` Device on which to store the parameter.

`dtype: torch.dtype | None` Data type of the parameter.

Store the embedding table as an `nn.Parameter` of shape $(\text{num_embeddings}, \text{embedding_dim})$. The forward method must accept integer token IDs of arbitrary shape and return that same shape with `embedding_dim` appended as the final dimension. Do not use `nn.Embedding` or `nn.functional.embedding`.

Initialize the table using the prescription in [Section 3.3.1](#).

To test your implementation, complete `adapters.run_embedding` and run `uv run pytest -k test_embedding`. The adapter should instantiate your module, load the supplied embedding table, and call the module.

3.4 Pre-Norm Transformer Block

3.4.1 Root Mean Square Layer Normalization

Each Transformer block has two sub-layers: a self-attention mechanism and a position-wise feed-forward network.

In the original Transformer paper [\[A. Vaswani et al., 2017\]](#), the model uses a residual connection around each of the two sub-layers, followed by layer normalization. This architecture is commonly known as the post-norm Transformer, since layer normalization is applied to the sub-layer output. However, a variety of work [\[T. Q. Nguyen et al., 2019\]](#); [\[R. Xiong et al., 2020\]](#) has found that moving layer normalization from the output of each sub-layer to the input of each sub-layer (with an additional layer normalization after the final Transformer block) improves Transformer training stability. The output of each Transformer block sub-layer is then added to the sub-layer input via the residual connection. An intuition for pre-norm is that there is a clean residual stream without any normalization going from the input embeddings to the final output of the Transformer, which is purported to improve gradient flow. This pre-norm Transformer is now the standard used in language models today, so we will implement this variant. We will walk through each of the components of a pre-norm Transformer block, implementing them in sequence.

The original Transformer uses layer normalization [\[J. L. Ba et al., 2016\]](#) to normalize activations. Following LLaMA [\[H. Touvron et al., 2023\]](#), we will use root mean square layer normalization (RMSNorm) [\[B. Zhang et al., 2019\]](#). Given a vector $x \in \mathbb{R}^{d_{\text{model}}}$ of activations, RMSNorm rescales each coordinate x_ℓ as follows:

$$\text{RMSNorm}(x)_\ell = \gamma_\ell \frac{x_\ell}{\text{RMS}(x)}, \quad \text{RMS}(x) = \sqrt{\frac{1}{d_{\text{model}}} \sum_{c=1}^{d_{\text{model}}} x_c^2 + \varepsilon_{\text{norm}}}.$$

Here, $\gamma \in \mathbb{R}^{d_{\text{model}}}$ is a learnable gain vector initialized to $\mathbf{1}$, and $\varepsilon_{\text{norm}}$ is a hyperparameter fixed at $1\text{e-}5$ for the final model. RMSNorm has no centering term and no bias. Perform the RMSNorm arithmetic in at least `torch.float32` precision: if the input uses a lower-precision floating-point dtype such as `torch.float16` or `torch.bfloat16`, temporarily cast it to `torch.float32`; if it is already `torch.float32` or higher precision, preserve that precision. Cast the final result back to the original input dtype. Overall, your `forward` method should look like:

```
in_dtype = x.dtype
if x.dtype in (torch.float16, torch.bfloat16):
    x = x.to(torch.float32)

# Your code here performing RMSNorm
...
result = ...

# Return the result in the original dtype
return result.to(in_dtype)
```

The block uses pre-normalization:

$$u = x + \text{Attention}(\text{RMSNorm}_{\text{attn}}(x)), \quad y = u + \text{FFN}(\text{RMSNorm}_{\text{ffn}}(u)).$$

Problem: Root Mean Square Layer Normalization (4 points)

Deliverable: Implement `RMSNorm` as a `torch.nn.Module`. We recommend the following interface:

```
def __init__(
    self,
    d_model: int,
    norm_eps: float = 1e-5,
    device: torch.device | None = None,
    dtype: torch.dtype | None = None,
):
    ...

def forward(self, x: torch.Tensor) -> torch.Tensor:
    ...
```

`d_model: int` Size of the final hidden dimension.

`norm_eps: float` Constant added for numerical stability.

`device: torch.device | None` Device on which to store the gain.

`dtype: torch.dtype | None` Data type of the gain.

The forward method must accept an input of shape `(..., d_model)` and return a tensor of the same shape and floating-point dtype. Initialize the learned gain to `1`, normalize only the final dimension, and do not center the activations or add a bias. Perform the RMSNorm arithmetic in at least `torch.float32` precision. For `torch.float16` or `torch.bfloat16` inputs, temporarily cast the activations to `torch.float32`; preserve inputs that are already `torch.float32` or higher precision. Cast the final result back to the original input dtype.

To test your implementation, complete `adapters.run_rmsnorm` and run `uv run pytest -k rmsnorm`. The adapter should instantiate your module, load the supplied gain vector, and call the module.

3.4.2 Position-Wise Feed-Forward Network

In the original Transformer paper [A. Vaswani et al., 2017], the Transformer feed-forward network consists of two linear transformations with a ReLU activation, $\text{ReLU}(x) = \max(0, x)$, between them. In that original architecture, the dimensionality of the inner feed-forward layer is typically four times the input dimensionality.

However, modern language models tend to incorporate two main changes compared to this original design: they use a different activation function and employ a gating mechanism. Specifically, we will implement the SwiGLU activation function adopted in LMs like Llama 3 [A. Grattafiori et al., 2024] and Qwen 2.5 [A. Yang et al., 2024], which combines the SiLU (often called Swish) activation with a gating mechanism called a Gated Linear Unit (GLU). We will also omit the bias terms sometimes used in linear layers, following most modern LMs since PaLM [A. Chowdhery et al., 2022] and LLaMA [H. Touvron et al., 2023].

The SiLU activation function [S. Elfwing et al., 2017], independently popularized as Swish by [P. Ramachandran et al., 2017] is defined as follows:

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}.$$

The SiLU activation function is similar to the ReLU activation function, but is smooth at zero. Figure 2 compares their shapes: unlike ReLU, SiLU permits small negative outputs and transitions smoothly through the origin.

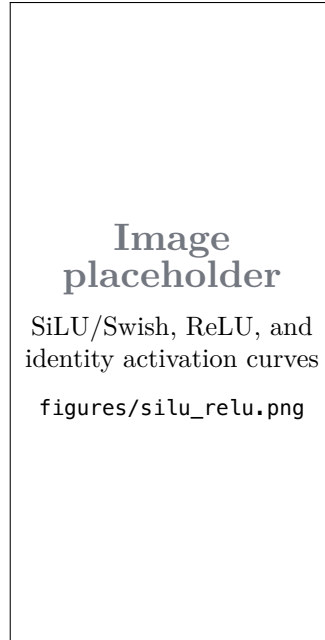


Figure 2: SiLU (aka Swish) compared with ReLU and the identity function.

Gated Linear Units were originally defined by [Y. N. Dauphin et al., 2017] as the element-wise product of a linear transformation passed through a sigmoid function and another linear transformation:

$$\text{GLU}(x, W_{\text{gate}}, W_{\text{up}}) = \sigma(W_{\text{gate}}x) \odot W_{\text{up}}x,$$

where $\odot$ represents element-wise multiplication. Gated Linear Units are suggested to reduce the vanishing gradient problem for deep architectures by providing a linear path for the gradients while retaining non-linear capabilities.

Putting the SiLU/Swish and GLU together, we get the SwiGLU [N. Shazeer, 2020], which we will use for our feed-forward networks:

$$\text{SwiGLU}(x) = W_{\text{down}} [\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x].$$

Here $x \in \mathbb{R}^{d_{\text{model}}}$, $W_{\text{gate}}, W_{\text{up}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, $W_{\text{down}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, and canonically $d_{\text{ff}} = \frac{8}{3}d_{\text{model}}$, which approximately matches the parameter count of a conventional two-matrix feed-forward network with width $4d_{\text{model}}$. For concrete implementations, this width is often rounded to a nearby multiple of 64 for hardware efficiency.

And to answer why SwiGLU? It's good to keep an empirical perspective: a now famous quote from Shazeer's paper is

“We offer no explanation as to why these architectures seem to work; we attribute their success, as all else, to divine benevolence.”

Problem: Implement the position-wise feed-forward network (4 points)

Deliverable: Implement the SiLU activation and the SwiGLU feed-forward network. You may use `torch.sigmoid` when implementing SiLU, but you may not use a supplied SiLU or gated feed-forward module.

Your SwiGLU module must contain three bias-free linear transformations. The gate and up projections map from `d_model` to `d_ff`; the down projection maps from `d_ff` back to `d_model`. It must accept

inputs of shape `(..., d_model)` and return outputs of the same shape. For the fixed model, use `d_model=512` and `d_ff=1344`.

To test your implementation, complete `adapters.run_silu` and `adapters.run_swiglu`. Then run `uv run pytest -k "silu or swiglu"`. The SwiGLU adapter should load all three supplied projection matrices into your module before calling it.

3.4.3 Relative Positional Embeddings

To inject positional information into the model, we will implement Rotary Position Embeddings [J. Su et al., 2021], often called RoPE. RoPE acts on each projected query and key head independently. For a projected query head $q^{(i)} \in \mathbb{R}^{d_h}$ at token position i , we apply a position-dependent rotation matrix R_i ,

$$q'^{(i)} = R_i q^{(i)}.$$

The same construction is applied to key heads, while value heads are left unchanged.

Under the adjacent-pair convention used in this assignment, the coordinate pairs

$$(q_1^{(i)}, q_2^{(i)}), (q_3^{(i)}, q_4^{(i)}), \dots, (q_{d_h-1}^{(i)}, q_{d_h}^{(i)})$$

are treated as independent two-dimensional vectors. For pair k , define the angular frequency ω_k and the rotation angle at position i by

$$\omega_k = \Theta^{-\frac{(2k-2)}{d_h}}, \quad \phi_{i,k} = i\omega_k, \quad k \in \{1, \dots, \frac{d_h}{2}\},$$

where Θ is the fixed RoPE frequency base. Let

$$R(\phi) = \begin{pmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{pmatrix}.$$

The full rotation at position i is therefore the block-diagonal matrix

$$R_i = \text{diag}\left(R(\phi_{i,1}), R(\phi_{i,2}), \dots, R(\phi_{i, \frac{d_h}{2}})\right) \in \mathbb{R}^{d_h \times d_h}.$$

Thus, RoPE does not add a positional vector to the hidden state. Instead, it rotates each two-dimensional coordinate pair of Q and K by a position-dependent angle. RoPE has no learnable parameters.

The exact same rotation process applied to $q^{(i)}$ is applied to $k^{(j)}$, using R_j . Figure 3 illustrates how a query and two keys at different token positions are rotated within one two-dimensional coordinate pair.

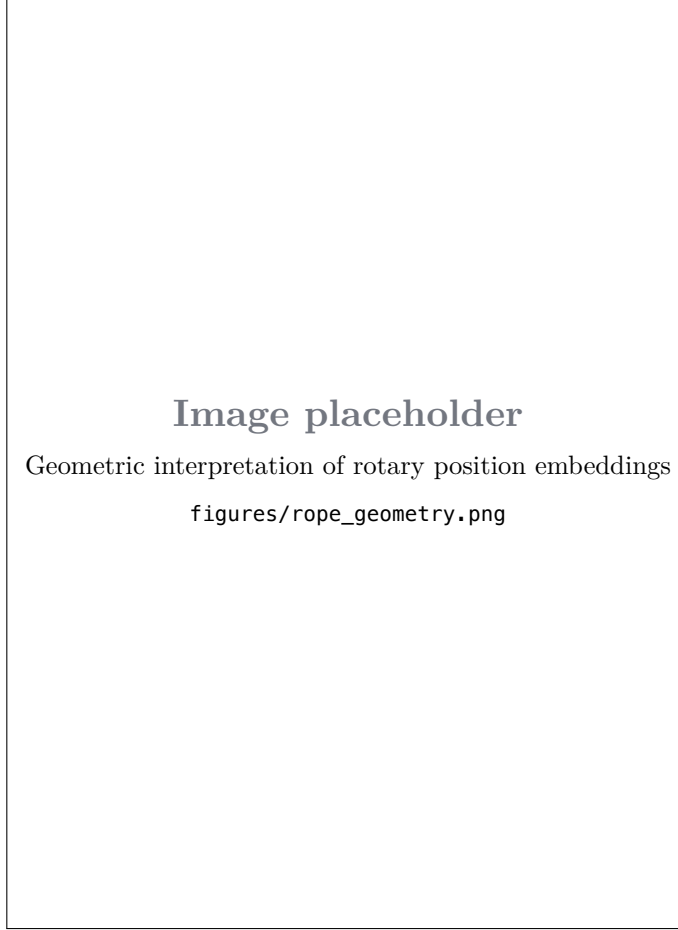


Figure 3: For a token sequence, we illustrate three tokens, q , k_1 , and k_2 , located at positions m , n_1 , and n_2 . For each token, we highlight three corresponding *chunks*: two-dimensional slices of the embedding to which RoPE applies independent rotations. Assuming the corresponding chunks of k_1 and k_2 are identical, RoPE rotates them by different angles because their positions n_1 and n_2 differ. Each chunk k is rotated at angular frequency ω_k , producing position-dependent inner products between q and each key.

The important consequence is that the dot product between a rotated query and key depends on their *relative* displacement. For each coordinate pair,

$$R(\phi_{i,k})^\top R(\phi_{j,k}) = R(\phi_{j,k} - \phi_{i,k}) = R((j-i)\omega_k).$$

Consequently,

$$(R_i q^{(i)})^\top (R_j k^{(j)}) = (q^{(i)})^\top R_i^\top R_j k^{(j)},$$

and the positional contribution to the query-key inner product is a function of the relative offset $j-i$.

Computationally efficient realization. The block-diagonal matrix R_i is sparse: every output coordinate depends on only the two coordinates in its own pair. Constructing a dense $d_h \times d_h$ matrix and multiplying by it would therefore do unnecessary work. Following the efficient realization in the original RoPE formulation [J. Su et al., 2021], we can write the same rotation using element-wise operations.

For any head vector

$$x = (x_1, x_2, \dots, x_{d_h}) \in \mathbb{R}^{d_h},$$

define the adjacent-pair quarter-turn

$$\text{rotate_pair}(x) = (-x_2, x_1, -x_4, x_3, \dots, -x_{d_h}, x_{d_h-1}).$$

At position i , define

$$c_i = (\cos \phi_{i,1}, \cos \phi_{i,1}, \cos \phi_{i,2}, \cos \phi_{i,2}, \dots, \cos \phi_{i, \frac{d_h}{2}}, \cos \phi_{i, \frac{d_h}{2}}),$$

$$s_i = (\sin \phi_{i,1}, \sin \phi_{i,1}, \sin \phi_{i,2}, \sin \phi_{i,2}, \dots, \sin \phi_{i, \frac{d_h}{2}}, \sin \phi_{i, \frac{d_h}{2}}).$$

Then the dense matrix multiplication $R_i x$ is exactly equivalent to

$$R_i x = x \odot c_i + \text{rotate_pair}(x) \odot s_i$$

where $\odot$ denotes element-wise multiplication. Equivalently, for pair k ,

$$x'_{2k-1} = x_{2k-1} \cos \phi_{i,k} - x_{2k} \sin \phi_{i,k},$$

$$x'_{2k} = x_{2k-1} \sin \phi_{i,k} + x_{2k} \cos \phi_{i,k}.$$

These equations are the same pairwise rotations encoded by R_i ; they simply avoid materializing the sparse rotation matrix.

For implementation, precompute the sine and cosine values for every supported token position and every adjacent coordinate pair, giving two tables of shape

$$[n_{\max}, \frac{d_h}{2}].$$

These values are fixed rather than learned, so store them as non-persistent module buffers, for example with `self.register_buffer(..., persistent=False)`, rather than as `nn.Parameters`. This allows the tables to move automatically with the module between devices without treating them as trainable parameters. During the forward pass, use `token_positions` to index the corresponding rows of the sine and cosine tables. Then split the final `head_dim` axis into adjacent coordinate pairs and apply the pairwise rotation above using vectorized tensor operations. The duplicated vectors c_i and s_i are therefore only a mathematical way to express the operation: in code, the compact `head_dim / 2` sine and cosine values can be broadcast directly across the two coordinates of each pair.

Another common RoPE implementation uses the *half-split* coordinate convention used in the GPT-NeoX lineage [S. Black et al., 2022]. It represents the same family of two-dimensional rotations under a different fixed coordinate arrangement. **The conventions must not be mixed.** In this assignment, implement the adjacent-pair convention defined above, which is the convention exercised by the tests.

Problem: Implement RoPE (6 points)

Deliverable: Implement a `RotaryPositionalEmbedding` class that applies the adjacent-pair RoPE convention defined above. We recommend the following interface:

```
def __init__(
    self,
    rope_theta: float,
    head_dim: int,
    context_length: int,
    device: torch.device | None = None,
):
    ...

def forward(
    self,
    x: torch.Tensor,
    token_positions: torch.Tensor,
) -> torch.Tensor:
```


...

rope_theta: `float` Frequency base Θ used by RoPE.

head_dim: `int` Dimension of each query and key head; it must be even.

context_length: `int` Number of supported token positions, equivalently the maximum supported sequence length.

device: `torch.device` | `None` Device on which to create the buffers.

The forward method must accept x with shape $(\dots, \text{sequence_length}, \text{head_dim})$ and integer `token_positions` with final dimension `sequence_length`. Any leading dimensions of `token_positions` must be broadcastable across the batch-like dimensions of x ; singleton dimensions may be inserted as needed to broadcast over attention-head axes. Return a tensor with the same shape and floating-point dtype as x .

Precompute the sine and cosine values for the $\frac{d_h}{2}$ adjacent coordinate pairs in buffers of shape $(\text{context_length}, \text{head_dim} / 2)$. During the forward pass, use `token_positions` to select the corresponding rows and apply the computationally efficient pairwise realization of

$$R_i x = x \odot c_i + \text{rotate_pair}(x) \odot s_i.$$

Do not construct or multiply by a dense rotation matrix. The stored sine and cosine buffers should remain in their compact $\text{head_dim} / 2$ form; any duplication across the two coordinates of each pair may be performed implicitly through slicing, reshaping, or broadcasting.

Reject odd head dimensions, mismatched final dimensions, non-integer positions, and positions outside $[0, \text{context_length})$.

To test your implementation, complete `adapters.run_rope` and run `uv run pytest tests/test_rope.py`.

3.4.4 Scaled Dot-Product Attention

We will now implement scaled dot-product attention as described by [A. Vaswani et al., 2017]. As a preliminary step, the definition of the attention operation will make use of softmax, an operation that takes an unnormalized vector of scores and turns it into a normalized distribution:

For a vector $z = (z_1, \dots, z_m) \in \mathbb{R}^m$, where m denotes the number of elements of z , softmax is defined coordinate-wise by

$$\text{softmax}(z)_\ell = \frac{\exp(z_\ell)}{\sum_{c=1}^m \exp(z_c)}, \quad \ell \in \{1, \dots, m\}.$$

Note that $\exp(z_\ell)$ can become infinite for large values, after which dividing infinity by infinity produces `NaN`. We can avoid this by noticing that the softmax operation is invariant to adding any constant c to all inputs. We can leverage this property for numerical stability—typically, we subtract the largest entry of z from all elements of z , making the new largest entry zero.

Let

$$z_{\max} = \max_u z_u$$

Then the numerically stable form is

$$\text{softmax}(z)_\ell = \frac{\exp(z_\ell - z_{\max})}{\sum_{c=1}^m \exp(z_c - z_{\max})}.$$

Scaled dot-product attention is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V.$$

Here, $Q \in \mathbb{R}^{n_q \times d_k}$, $K \in \mathbb{R}^{n_{kv} \times d_k}$, and $V \in \mathbb{R}^{n_{kv} \times d_v}$, where n_q is the number of query positions and n_{kv} is the common number of key and value positions. The Q , K , and V in this equation are inputs to the

operation; they are not themselves the learnable parameters. For self-attention in this model, $n_q = n_{kv} = n$ and $d_k = d_v = d_h$.

It is sometimes convenient to mask the attention scores before softmax. A mask may have shape $M \in \{\text{True}, \text{False}\}^{n_q \times n_{kv}}$ or additional leading dimensions that are broadcastable to the leading batch-like dimensions of the score tensor. In this assignment, a value of **True** at position (i, j) means that query i may attend to key j , while **False** means that the query-key pair is masked.

Computationally, it is much more efficient to use masking than to compute attention on subsequences. We do this by taking the pre-softmax values $\frac{QK^\top}{\sqrt{d_k}}$ and replacing any entry whose mask value is **False** with negative infinity before applying softmax. The implementation should accept arbitrary leading batch-like dimensions.

Problem: Implement softmax (2 points)

Deliverable: Write a function that accepts a tensor and a dimension `dim`, and applies softmax along that dimension. The output must have the same shape as the input, with values along `dim` forming normalized probability distributions. Subtract the maximum value along `dim` before exponentiating so the computation remains numerically stable for large inputs. To test your implementation, complete `adapters.run_softmax` and run `uv run pytest -k softmax`.

Problem: Implement scaled dot-product attention (5 points)

Deliverable: Implement scaled dot-product attention. Queries and keys have shapes $(\dots, n_q, d_k)$ and $(\dots, n_{kv}, d_k)$; values have shape $(\dots, n_{kv}, d_v)$. The result must have shape $(\dots, n_q, d_v)$. The leading batch-like dimensions must broadcast, the query and key sequence lengths may differ, and d_v need not equal d_k . Your implementation should also support an optional user-provided Boolean mask whose final two dimensions are (n_q, n_{kv}) and whose leading dimensions, if present, are broadcastable to the leading batch-like dimensions of the attention scores. For each query, the probabilities over its permitted key positions must sum to one, while probabilities at positions with a mask value of **False** must be zero. Validate that a supplied mask defines a valid probability distribution for every query, and fail clearly rather than returning non-finite values when masked softmax would be undefined. To test your implementation, complete `adapters.run_scaled_dot_product_attention` and run `uv run pytest -k scaled_attention`.

3.4.5 Causal Grouped-Query Self-Attention

We begin from ordinary multi-head self-attention (MHA). For h_q attention heads,

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_{h_q}),$$

$$\text{head}_a = \text{Attention}(Q_a, K_a, V_a), \quad a \in \{1, \dots, h_q\},$$

where each head has dimension d_h . The corresponding self-attention sublayer is

$$\text{MultiHeadSelfAttention}(x) = W_O \text{MultiHead}(W_Q x, W_K x, W_V x).$$

In ordinary MHA,

$$W_Q, W_K, W_V \in \mathbb{R}^{h_q d_h \times d_{\text{model}}}, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times h_q d_h}.$$

The output dimensions of W_Q , W_K , and W_V are therefore naturally interpreted as concatenated head dimensions.

Causal masking. Self-attention in an autoregressive language model must not use future tokens. For a sequence $x_1, \dots, x_n$, the representation at query position i may depend only on key positions $j \leq i$. Positions $j > i$ are therefore masked before the softmax. This is the same Boolean convention introduced in

[Section 3.4.4](#): allowed query-key pairs are **True**, while future positions are **False**. You can construct the causal mask with `torch.triu` or a broadcasted comparison between query and key indices.

Applying RoPE. Apply RoPE after projecting and reshaping queries and keys into heads, and before computing their attention scores. Values are not rotated. The axis indexing attention heads is batch-like—the same RoPE operation is applied independently to every head—while the final d_h axis remains the feature dimension on which each rotation acts. The attention forward interface should be `attention(x, token_positions=None)`. If `token_positions` is **None**, use `0, ..., sequence_length - 1`; otherwise, use the supplied positions when rotating both Q and K .

Grouped-query attention. Grouped-query attention (GQA) [\[J. Ainslie et al., 2023\]](#) keeps h_q query heads but uses only h_{kv} key heads and h_{kv} value heads. We require d_{model} to be divisible by h_q and h_q to be divisible by h_{kv} . Define

$$g = \frac{h_q}{h_{kv}},$$

so each key/value head is shared by a group of g query heads. [Figure 4](#) contrasts this sharing pattern with ordinary multi-head and multi-query attention. For the fixed model in this assignment, $h_q = 16$, $h_{kv} = 4$, and hence $g = 4$.



Figure 4: Overview of the grouped-query method. Multi-head attention has h_q query, key, and value heads. Multi-query attention shares a single key head and a single value head across all query heads. Grouped-query attention instead shares one key head and one value head within each group of query heads, interpolating between multi-head and multi-query attention.

The projection matrices now have shapes

$$W_Q \in \mathbb{R}^{h_q d_h \times d_{\text{model}}}, \quad W_K, W_V \in \mathbb{R}^{h_{kv} d_h \times d_{\text{model}}}, \quad W_O \in \mathbb{R}^{d_{\text{model}} \times h_q d_h}.$$

After projection, keep the g query heads that share a key/value head together. Query head $a = (h - 1)g + r$ belongs to key/value head h , where $r \in \{1, \dots, g\}$. Arrange the tensors as follows:

Tensor	Meaning	Shape
Q	grouped query heads	$[B, h_{kv}, g, n, d_h]$
K, V	key/value heads	$[B, h_{kv}, n, d_h]$
S	attention scores	$[B, h_{kv}, g, n, n]$
A	attention probabilities	$[B, h_{kv}, g, n, n]$
O	grouped attention output	$[B, h_{kv}, g, n, d_h]$

In the equations below, Q and K denote the projected, RoPE-rotated query and key tensors. For each batch element b , key/value head h , query head within that group r , query position i , and key position j , compute

the scaled score

$$S_{b,h,r,i,j} = \frac{1}{\sqrt{d_h}} \sum_{\ell=1}^{d_h} Q_{b,h,r,i,\ell} K_{b,h,j,\ell}.$$

Apply the causal mask before softmax,

$$\tilde{S}_{b,h,r,i,j} = \begin{cases} S_{b,h,r,i,j}, & j \leq i, \\ -\infty, & j > i, \end{cases}$$

and normalize over the key-position index j :

$$A_{b,h,r,i,:} = \text{softmax}(\tilde{S}_{b,h,r,i,:}).$$

Finally, contract the attention probabilities with the shared value head,

$$O_{b,h,r,i,\ell} = \sum_{j=1}^n A_{b,h,r,i,j} V_{b,h,j,\ell}.$$

This contraction shares each K_h, V_h pair across its g query heads by broadcasting; do not materialize repeated key or value heads. Afterward, merge the (h, r) axes back into the original query-head order, giving

$$[B, h_{kv}, g, n, d_h] \longrightarrow [B, n, h_q d_h] = [B, n, d_{\text{model}}],$$

and apply the output projection W_O .

The same implementation covers all three common cases: MHA has $h_{kv} = h_q$ and therefore $g = 1$; GQA has $1 < h_{kv} < h_q$; and MQA has $h_{kv} = 1$ and therefore $g = h_q$.

Low-Resource Tip

Keep queries as $[B, h_{kv}, g, n, d_h]$. An einsum over d_h naturally broadcasts each key/value head across the g query heads in its group. When merging heads back together, preserve the same (h, r) ordering implied by $a = (h-1)g + r$; otherwise the output projection will receive the query heads in a different order from the projection layout.

Problem: Implement causal grouped-query self-attention (10 points)

Deliverable: Implement causal, RoPE-enabled grouped-query self-attention as a `torch.nn.Module`. Your implementation should accept, at minimum, the following construction parameters:

`d_model`: `int` Dimension of the input and output residual stream.

`n_q_heads`: `int` Number of query heads.

`n_kv_heads`: `int` Number of key and value heads.

`context_length`: `int` Maximum supported sequence length.

`rope_theta`: `float` RoPE frequency base.

Its forward interface should follow:

```
def forward(
    self,
    x: torch.Tensor,
    token_positions: torch.Tensor | None = None,
) -> torch.Tensor:
    ...
```

The input `x` has shape `(batch_size, sequence_length, d_model)` and the output must have the same shape. If supplied, `token_positions` must have final dimension `sequence_length`, with leading dimensions broadcastable to the batch-like dimensions of `x`; both `(sequence_length,)` and `(batch_size,`

`sequence_length`) are valid. Require `d_model` to be divisible by `n_q_heads`, and `n_q_heads` to be divisible by `n_kv_heads`. Project all query heads in one matrix multiplication and all key heads and value heads in one matrix multiplication each. Apply adjacent-pair RoPE to queries and keys, but never to values. If `token_positions` is `None`, use `0, ..., sequence_length - 1`; otherwise use the supplied positions for both queries and keys. Apply a causal mask that permits key position j if and only if $j \leq i$, compute the grouped contractions above without materializing repeated key or value heads, restore the original query-head order, and apply the output projection. The same implementation must support MHA (`n_kv_heads == n_q_heads`), GQA, and MQA (`n_kv_heads == 1`). To test your implementation, complete `adapters.run_grouped_query_self_attention` and run `uv run pytest -k gqa`. The adapter should construct your attention module, load the supplied query, key, value, and output projection matrices, and pass through supplied token positions when present.

3.5 The Full Transformer LM

Integrate the components above into the complete model. Each block contains two independent RMSNorms, causal RoPE-GQA, SwiGLU, and two residual additions. The block forward interface is `block(x, token_positions=None)`: any supplied token positions are passed unchanged to the attention sublayer, while the feed-forward sublayer is position-wise and does not use them. The full language model adds the learned token embedding, a final RMSNorm, and the untied, bias-free LM head described in [Section 3.1](#). In an ordinary full-model forward pass, the blocks may be called without explicit token positions, so attention uses the default contiguous positions. Do not add absolute position embeddings; RoPE operates inside attention. Reject sequences longer than `context_length`. With the fixed architecture, the complete model must contain exactly 19,272,192 trainable parameters.

Problem: Implement the Transformer block (5 points)

Deliverable: Implement the pre-norm Transformer block described in [Section 3.4](#) and illustrated in [Figure 1](#). Your block should accept, at minimum, `d_model`, `n_q_heads`, `n_kv_heads`, `d_ff`, `context_length`, `rope_theta`, and `norm_eps` as construction parameters. The attention and feed-forward sublayers must use distinct RMSNorm modules and independent parameters. Apply RMSNorm before each sublayer and place each sublayer inside its own residual connection. The block forward interface should follow:

```
def forward(
    self,
    x: torch.Tensor,
    token_positions: torch.Tensor | None = None,
) -> torch.Tensor:
    ...
```

The input `x` has shape `(batch_size, sequence_length, d_model)` and the block must return the same shape. If supplied, `token_positions` must have final dimension `sequence_length`, with leading dimensions broadcastable to the batch-like dimensions of `x`. Pass `token_positions` unchanged to the attention sublayer. If it is `None`, the attention module uses its default contiguous positions; the SwiGLU sublayer does not use token positions.

To test your implementation, complete `adapters.run_transformer_block` and run `uv run pytest -k transformer_block`. The adapter should translate the canonical fixture names if necessary, load every supplied parameter, and call your block.

Problem: Implement the Transformer LM (6 points)

Deliverable: Implement the complete Transformer language model from [Section 3.5](#). At minimum, its constructor must accept `vocab_size`, `context_length`, `d_model`, `num_layers`, `n_q_heads`, `n_kv_heads`, `d_ff`, `rope_theta`, and `norm_eps`. Although the final architecture is fixed by [Section 3](#), do not hardcode these dimensions: the tests use smaller valid models. Store the constructor value as `self.context_length = context_length` so downstream utilities can query the model's supported context window.

Its forward interface should follow:

```
def forward(
    self,
    token_ids: torch.Tensor,
    token_positions: torch.Tensor | None = None,
) -> torch.Tensor:
    ...
```

The forward method accepts token IDs of shape `(batch_size, sequence_length)` and returns unnormalized logits of shape `(batch_size, sequence_length, vocab_size)`. If supplied, `token_positions` must have final dimension `sequence_length`, with leading dimensions broadcastable to the batch-like dimensions of `token_ids`; both `(sequence_length,)` and `(batch_size, sequence_length)` are valid. Pass supplied positions unchanged to every Transformer block. If `token_positions` is `None`, each attention module uses the default contiguous positions. Require `1 <= sequence_length <= context_length`; reject empty sequences and sequences longer than `context_length`. Use a token embedding, `num_layers` independently parameterized blocks, a final RMSNorm, and an untied bias-free LM head. Do not add absolute positional embeddings, apply softmax inside the model, or introduce dropout.

To test your implementation, complete `adapters.run_transformer_lm` and `adapters.get_transformer_lm`. Then run `uv run pytest tests/test_model.py`. The first adapter should load every supplied fixture weight; the second must return the actual student `torch.nn.Module` so the tests can inspect parameters and gradients.

4 Loss and Optimization

We now have the tokenized data from [Section 2.4](#) and the model from [Section 3](#). What remains in this section is to define the loss and optimization utilities used during training:

- Loss: the cross-entropy objective used to train and evaluate the language model.
- Optimizer: AdamW, together with the learning-rate schedule and gradient clipping used for each update.

After implementing these pieces, we combine them with data loading, checkpointing, and the training loop in [Section 5](#).

4.1 Cross-Entropy and Perplexity

Recall that the Transformer language model defines a distribution $p_{\theta}(x_{i+1} \mid x_{1:i})$ for each sequence x of length $n + 1$ and $i = 1, \dots, n$. Given a training set $\mathcal{D}$ consisting of sequences of length $n + 1$, we define the standard cross-entropy (negative log-likelihood) objective:

$$\mathcal{L}(\theta; \mathcal{D}) = \frac{1}{|\mathcal{D}|n} \sum_{x \in \mathcal{D}} \sum_{i=1}^n -\log p_{\theta}(x_{i+1} \mid x_{1:i}).$$

A single Transformer forward pass yields logits for all $i = 1, \dots, n$ in parallel; these logits parameterize the conditionals $p_\theta(x_{i+1} \mid x_{1:i})$. In particular, let $z_i \in \mathbb{R}^{|V|}$ denote the logits at position i . Then

$$p_\theta(x_{i+1} \mid x_{1:i}) = \text{softmax}(z_i)[x_{i+1}] = \frac{\exp(z_i[x_{i+1}])}{\sum_{c=1}^{|V|} \exp(z_i[c])}.$$

Implementing the cross-entropy loss requires some care with numerical issues, just as in the case of softmax. For target y and logits z ,

$$\ell(z, y) = -z_y + \log \sum_{c=1}^{|V|} e^{z_c}.$$

Evaluate the log-sum-exp with a max shift. Average over all batch and sequence positions. You may use `torch.logsumexp`.

Perplexity. Cross-entropy suffices for training, but when we evaluate the model, we also want to report perplexity. For a sequence of length n where we incur per-token cross-entropy losses $\ell_1, \dots, \ell_n$:

$$\text{perplexity} = \exp \left(\frac{1}{n} \sum_{i=1}^n \ell_i \right).$$

Problem: Implement cross-entropy (3 points)

Deliverable: Write a function that accepts predicted logits of shape `(..., vocab_size)` and integer targets of shape `(...)`. Compute the negative log-likelihood of the target class at every position and return the mean over all leading dimensions.

Subtract the largest logit before exponentiating, or use an equivalent stable log-sum-exp computation. You may use `torch.logsumexp`. Avoid materializing a full `log(softmax(logits))` tensor when the target logits can be selected directly. For this deliverable, do not use `torch.softmax`, `torch.log_softmax`, `torch.nn.functional.cross_entropy`, or another supplied cross-entropy implementation.

To test your implementation, complete `adapters.run_cross_entropy` and run `uv run pytest -k cross_entropy`.

4.2 AdamW

Modern language models are typically trained with more sophisticated optimizers than SGD. Most optimizers used recently are derivatives of the Adam optimizer [D. P. Kingma et al., 2015]. We will use AdamW [I. Loshchilov et al., 2019], which is in wide use in recent work. AdamW proposes a modification to Adam that improves regularization by adding weight decay (at each update, we pull the parameters toward zero) in a way that is decoupled from the gradient update.

AdamW is stateful: for each parameter, it keeps track of a running estimate of its first and second moments. Thus, AdamW uses additional memory in exchange for improved stability and convergence. For a run of N_{updates} optimizer updates, the global optimization step is $s \in \{0, \dots, N_{\text{updates}} - 1\}$. At step s , AdamW receives the current scheduler-provided learning rate α_s . It also has a pair of hyperparameters (β_1, β_2) that control the updates to the moment estimates, and a weight-decay rate λ . Typical applications set (β_1, β_2) to $(0.9, 0.999)$, but large language models like LLaMA [H. Touvron et al., 2023] and GPT-3 [T. B. Brown et al., 2020] are often trained with $(0.9, 0.95)$. A small value $\varepsilon_{\text{Adam}}$, such as 10^{-8} , improves numerical stability when we obtain extremely small values in the second moment.

Algorithm: AdamW Optimizer

1	$\text{init}(\theta_0)$	▷ Initialize learnable parameters
2	$\mathbf{m}_0 \leftarrow 0$	▷ Initialize first-moment estimate; same shape as θ_0
3	$\mathbf{v}_0 \leftarrow 0$	▷ Initialize second-moment estimate; same shape as θ_0
4	for $s = 0, \dots, N_{\text{updates}} - 1$ do	
5	Sample minibatch $\mathcal{B}_s$	
6	$\mathbf{g}_s \leftarrow \nabla_{\theta_s} \mathcal{L}(\theta_s; \mathcal{B}_s)$	▷ Compute gradient of the loss
7	$\mathbf{m}_{s+1} \leftarrow \beta_1 \mathbf{m}_s + (1 - \beta_1) \mathbf{g}_s$	▷ Update first-moment estimate
8	$\mathbf{v}_{s+1} \leftarrow \beta_2 \mathbf{v}_s + (1 - \beta_2)(\mathbf{g}_s \odot \mathbf{g}_s)$	▷ Update second-moment estimate
9	$\widehat{\mathbf{m}}_{s+1} \leftarrow \frac{\mathbf{m}_{s+1}}{1 - \beta_1^{s+1}}$	▷ Correct first-moment initialization bias
10	$\widehat{\mathbf{v}}_{s+1} \leftarrow \frac{\mathbf{v}_{s+1}}{1 - \beta_2^{s+1}}$	▷ Correct second-moment initialization bias
11	$\tilde{\theta}_{s+1} \leftarrow (1 - \alpha_s \lambda) \theta_s$	▷ Apply decoupled weight decay
12	$\theta_{s+1} \leftarrow \tilde{\theta}_{s+1} - \alpha_s \frac{\widehat{\mathbf{m}}_{s+1}}{\sqrt{\widehat{\mathbf{v}}_{s+1} + \varepsilon_{\text{Adam}}}}$	▷ Apply adaptive Adam update
13	end for	

AdamW optimizer **pseudocode**.

The displayed algorithm uses the global optimization step s and assumes every parameter receives a gradient on every update. A PyTorch optimizer must also handle parameters that are skipped on some updates. Maintain the moment tensors and an update count t_p separately for each parameter p that receives a gradient. Increment t_p only when that parameter is actually updated, and use t_p —not the global step s —in its bias-correction factors $1 - \beta_1^{t_p}$ and $1 - \beta_2^{t_p}$. The learning rate used for both decoupled weight decay and the adaptive update is the current value α_s stored in that parameter group.

The deliverable below specifies the PyTorch implementation contract, including parameter-group validation, lazy optimizer state, skipped gradients, closure handling, and serialization.

Problem: Implement AdamW (5 points)

Deliverable: Implement AdamW as a subclass of `torch.optim.Optimizer`. The following is the recommended interface and an otherwise empty template for a PyTorch optimizer:

```
class AdamW(torch.optim.Optimizer):
    def __init__(
        self,
        params,
        lr: float = 1e-3,
        betas: tuple[float, float] = (0.9, 0.999),
        eps: float = 1e-8,
        weight_decay: float = 0.0,
    ):
        ...

    def step(self, closure=None):
        ...
```

`params` Iterable of parameters or PyTorch-style parameter groups.

`lr`: **float** Current scheduler-provided learning rate α_s .
`betas`: **tuple[float, float]** Moment coefficients (β_1, β_2) .
`eps`: **float** Numerical-stability constant ϵ_{Adam} .
`weight_decay`: **float** Decoupled weight-decay coefficient λ .
 Require `lr` ≥ 0 , `eps` ≥ 0 , `weight_decay` ≥ 0 , and $0 \leq \text{beta1} < 1$, $0 \leq \text{beta2} < 1$. Validate the constructor defaults and, after calling the base-class constructor, validate the effective values in every parameter group so that group-specific overrides cannot bypass these constraints. Because callers may later modify a group's learning rate, read the current `lr`, `betas`, `eps`, and `weight_decay` from the group inside `step` and reject invalid effective values before applying an update.
 For every parameter with a dense gradient, lazily initialize `self.state[p]` with an update count of zero, `exp_avg` = `torch.zeros_like(p)`, and `exp_avg_sq` = `torch.zeros_like(p)`. Skip `grad` is `None` parameters completely, including weight decay and timestep advancement, and reject sparse gradients. Increment the parameter's update count before its bias correction, update the first and second moments, compute the corresponding bias corrections, apply decoupled weight decay directly to the parameter, and finally apply the bias-corrected adaptive update from the displayed algorithm. Perform all parameter updates without constructing an autograd graph.
 If `closure` is not **None**, evaluate it once with gradient recording enabled before performing any parameter updates, and return the resulting loss; otherwise `step` may return **None**. The inherited `state_dict` and `load_state_dict` methods must preserve the per-parameter update counts and moment tensors for an exact continuation of the optimizer state.
 To test your implementation, complete `adapters.get_adamw_cls` and run `uv run pytest -k adamw`.

4.3 Schedule and Gradient Clipping

The learning rate that works best often changes over the course of training. Transformer training commonly begins conservatively with a small learning rate, increases it during a warmup phase to a peak value, and then gradually decays it as training proceeds. In this assignment, we will use linear warmup followed by cosine annealing, following the schedule used in LLaMA [H. Touvron et al., 2023].

A scheduler is a function that takes the current global optimization step s and the schedule hyperparameters and returns the learning rate for that update. Our schedule is specified by (i) the current step s , (ii) the maximum learning rate $\alpha_{\max}$ reached at the end of warmup, (iii) the minimum learning rate $\alpha_{\min}$, (iv) the warmup endpoint s_w , and (v) the final cosine-decay step s_c .

Use linear warmup followed by cosine decay to a minimum learning rate. For global step s , maximum rate $\alpha_{\max}$, minimum rate $\alpha_{\min}$, warmup endpoint s_w , and cosine endpoint s_c ,

$$\alpha_s = \begin{cases} \frac{s}{s_w} \alpha_{\max}, & s < s_w, \\ \alpha_{\min} + \frac{1}{2} \left(1 + \cos \left(\pi \frac{s-s_w}{s_c-s_w} \right) \right) (\alpha_{\max} - \alpha_{\min}), & s_w \leq s \leq s_c, \\ \alpha_{\min}, & s > s_c. \end{cases}$$

During training, unusually large gradients can destabilize an update, so we will apply global gradient clipping after the backward pass and before the optimizer step. Compute

$$\|\mathbf{g}\|_2 = \sqrt{\sum_p \sum_{\ell} (g_{\ell}^{(p)})^2}.$$

If $\|\mathbf{g}\|_2 \leq M$, leave every gradient unchanged; otherwise, multiply every gradient by the same factor $\frac{M}{(\|\mathbf{g}\|_2 + \epsilon_{\text{clip}})}$, using a small ϵ_{clip} such as 10^{-6} for numerical stability.

Problem: Implement cosine learning-rate scheduling with warmup (2 points)

Write a function that accepts the current integer global step s , maximum learning rate $\alpha_{\max}$, minimum learning rate $\alpha_{\min}$, integer warmup endpoint s_w , and integer cosine endpoint s_c , and returns the learning rate defined above. Support zero warmup and hold the learning rate at $\alpha_{\min}$ after s_c . Require $0 \leq \alpha_{\min} \leq \alpha_{\max}$, and reject non-integer step indices, negative steps, and schedules for which $s_w < 0$ or $s_w \geq s_c$.

To test your implementation, complete `adapters.run_get_lr_cosine_schedule` and run `uv run pytest -k cosine_schedule`.

Problem: Implement gradient clipping (2 points)

Write a function that accepts an iterable of parameters and a maximum global ℓ_2 -norm. Ignore parameters whose gradient is `None`. Compute one norm over all remaining gradients and, only when it exceeds the maximum, multiply every gradient in place by the same scale factor. Return the norm before clipping as a Python `float`; return `0.0` if no parameter has a gradient. Reject a nonpositive maximum norm.

To test your implementation, complete `adapters.run_gradient_clipping` and run `uv run pytest -k gradient_clipping`.

5 Training Infrastructure and Loop

5.1 Data Loader

The tokenized training data is a single sequence of token IDs $u = (u_0, \dots, u_{N-1})$. Although the source dataset consists of separate stories, language-model training commonly concatenates the documents into one stream and places a delimiter such as `<|endoftext|>` between them. For TinyStories, the provided preprocessing produces

`encode(story1), EOT, encode(story2), EOT, ...`

with no BOS token, padding, truncation, or precomputed context windows. A sampled sequence may cross a document boundary; the EOT token marks that transition for the model.

A data loader turns the stream into minibatches. Each batch contains B input sequences of length n and their corresponding next-token targets, also of length n . This organization makes sampling simple because every start satisfying $0 \leq o < N - n$ defines a valid training example. Since every sampled sequence has the same length, no input padding is necessary, which avoids spending model compute on padding tokens and permits efficient dense batches. Sampling windows from one long stream also means the complete dataset does not need to reside in physical memory.

The tokenized files are raw little-endian unsigned 16-bit token IDs. Since $8192 < 2^{16}$, each ID occupies two bytes. The full training file is still too large to duplicate as an in-memory `int64` array. Unix-like systems provide the `mmap` system call, which maps a file on disk into a process's virtual address space and loads pages only when they are accessed. NumPy exposes this mechanism through `np.memmap`, allowing the file to behave like an array while the operating system reads only the portions requested by the sampler:

```
train = np.memmap(
    "data/tinystories/data/train.bin",
    mode="r",
    dtype=np.dtype("<u2"),
)
```

The dtype passed to `np.memmap` must match the file representation exactly. Opening with `mode="r"` prevents accidental modification of the course data. A small loader around this call should reject missing files and odd byte lengths, then return the one-dimensional, read-only memory map. Do not call `np.asarray(train, dtype=np.int64)` on the complete stream, since that allocates an eight-byte in-memory copy of every token and defeats the purpose of memory mapping.

Problem: Open a flat token stream (2 points)

Deliverable: Write a function that accepts a filesystem path and returns a one-dimensional, read-only `np.memmap` with dtype `np.dtype("<u2")`. The file has no header. Reject a missing file and a file with an odd byte length. Do not read or convert the complete stream into memory. To test your implementation, complete `adapters.run_load_token_array` and run `uv run pytest -k load_token_array`.

For a sampled start o , read $n + 1$ consecutive IDs and form

$$x = (u_o, \dots, u_{o+n-1}), \quad y = (u_{o+1}, \dots, u_{o+n}).$$

Sampling B starts gives tensors $x, y \in \mathbb{N}^{B \times n}$. The upper bound passed to `torch.randint` is exclusive, so using $N-n$ permits the last valid window without reading beyond the stream. Slice each $n + 1$ token window while it is backed by the memory map, stack only the sampled windows, convert that small batch to `torch.long`, and finally move it to the requested device.

Random sampling should be reproducible without repeatedly resetting PyTorch's global seed. A `torch.Generator` stores an explicit random-number-generator state, so `get_batch` should accept a generator argument and pass it to `torch.randint`. The training script creates and seeds its generator once; the tests create a generator and forward it through `adapters.run_get_batch`. Generators initialized with the same seed produce the same sequence of batches, while successive calls using one generator naturally advance to new batches.

Problem: Implement next-token batch sampling (4 points)

Deliverable: Write a function that accepts a one-dimensional token array, `batch_size`, `sequence_length`, a PyTorch device, and a `torch.Generator`. Sample `batch_size` starting positions uniformly from $[0, \text{len}(\text{dataset}) - \text{sequence_length})$ using that generator. Return the corresponding inputs and one-token-shifted targets as two `torch.long` tensors of shape $(\text{batch_size}, \text{sequence_length})$ on the requested device.

Require positive batch and sequence lengths and enough tokens to read at least one window of length `sequence_length + 1`. Read only the sampled slices from the token array, and convert only those slices to `torch.long`.

To test your implementation, complete `adapters.run_get_batch` and run `uv run pytest -k get_batch`.

Implementation note

Sampling through $N - n$ inclusively reads past the file. Sampling only n IDs leaves no final target. Converting the complete memory map to `int64` defeats its purpose.

5.2 Checkpointing

In addition to loading data, we will also need to save models as we train. When running jobs, we often want to be able to resume a training run that stopped midway through (e.g., due to your job timing out, machine failure, etc.). Even when all goes well, we might also want to later have access to intermediate models (e.g., to study training dynamics post-hoc, take samples from models at different stages of training, etc.).

A checkpoint should contain the state needed to continue the same training trajectory. Besides the model parameters and optimizer state, our batch samplers use stateful `torch.Generator` objects, so the training and validation generator states must also be saved and restored. We additionally store `next_step`, defined as the next global optimization step s that should execute after loading the checkpoint. Thus a checkpoint written immediately after completing step s stores `next_step = s + 1`; before any updates, `next_step = 0`.

PyTorch makes these pieces easy to serialize. Every `nn.Module` has a `state_dict()` method that returns its parameters and persistent buffers, and `torch.optim.Optimizer` provides the same state-dictionary interface for optimizer state. A `torch.Generator` exposes its RNG state through `get_state()` and can be restored with `set_state()`. Finally, `torch.save(obj, dest)` can serialize a dictionary containing tensors, integers, and other Python objects to a path or file-like object, and `torch.load(src)` can recover it later.

Problem: Implement model checkpointing (4 points)

Implement the following two functions to load and save checkpoints:

```
def save_checkpoint(
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    next_step: int,
    train_generator: torch.Generator,
    val_generator: torch.Generator,
    out: str | os.PathLike | typing.BinaryIO | typing.IO[bytes],
) -> None:
    ...
```

This function should save the model state, optimizer state, `next_step`, and the states of both generators into `out`. Use `state_dict()` for the model and optimizer and `get_state()` for each generator. You may store these pieces in a dictionary and serialize it with `torch.save(obj, out)`; PyTorch supports either a path or a file-like object.

This function expects the following parameters:

`model`: `torch.nn.Module`
`optimizer`: `torch.optim.Optimizer`
`next_step`: `int` Next global optimization step to execute after loading.
`train_generator`: `torch.Generator` Generator used for training batches.
`val_generator`: `torch.Generator` Independent generator used for validation batches.
`out`: `str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`

```
def load_checkpoint(
    src: str | os.PathLike | typing.BinaryIO | typing.IO[bytes],
    model: torch.nn.Module,
    optimizer: torch.optim.Optimizer,
    train_generator: torch.Generator,
    val_generator: torch.Generator,
) -> int:
    ...
```

This function should load a checkpoint from `src`, restore the model and optimizer with `load_state_dict()`, restore both generators with `set_state()`, and return the saved `next_step`. The caller can then resume the loop beginning exactly at that global step.

This function expects the following parameters:

`src`: `str | os.PathLike | typing.BinaryIO | typing.IO[bytes]`
`model`: `torch.nn.Module`
`optimizer`: `torch.optim.Optimizer`
`train_generator`: `torch.Generator`
`val_generator`: `torch.Generator`

Implement the `adapters.run_save_checkpoint` and `adapters.run_load_checkpoint` adapters, and make

```
sure they pass uv run pytest -k test_checkpointing.
```

5.3 Putting It Together

We will now put together the major components we have built so far: the data loader from [Section 5.1](#), the model from [Section 3.5](#), and the loss and optimization code from [Sections 4.1–4.3](#).

It will pay off to make it easy to start training runs with different hyperparameters (e.g., by taking them as command-line arguments), since you will use short runs to debug and may need to adjust settings to fit your available hardware. In particular, we recommend that your training script allow for at least the following:

- Ability to configure and control the various model and optimizer hyperparameters.
- Memory-efficient loading of large training and validation datasets with `np.memmap`.
- Periodically logging training and validation performance to the console.
- Saving model checkpoints so a useful run is not lost.

Create and seed two independent `torch.Generator` objects: one for training batches and one for validation batches. Keeping them separate ensures that changing the evaluation frequency does not change the future sequence of training minibatches. If a checkpoint is loaded, restore both generator states and set `next_step` to the value returned by `load_checkpoint`; otherwise start from `next_step = 0`.

5.3.1 Optimizer steps and gradient accumulation

A *global training step* means one optimizer update. It does not mean one forward pass or one sampled microbatch. Suppose a microbatch contains B sequences of length n and gradients are accumulated across n_{acc} microbatches. One optimizer step then uses

$$N_{\text{tok/update}} = B n n_{\text{acc}}$$

sampled tokens. The product $B n_{\text{acc}}$ is often called the *effective batch size* in sequences. Gradient accumulation lets you obtain a larger effective batch without storing all of its activations simultaneously, which is useful when a microbatch containing the full effective batch does not fit in GPU memory.

Call `optimizer.zero_grad()` once at the beginning of the global step. For each of the n_{acc} independently sampled microbatches, compute its ordinary mean cross-entropy, divide that loss by n_{acc} , and call `backward()`. Dividing by n_{acc} makes the accumulated gradient equal to the gradient of the mean of the n_{acc} microbatch losses. After all n_{acc} backward passes, apply global gradient clipping once and call `optimizer.step()` once. The learning-rate schedule, checkpoint field `next_step`, and final training budget all count these optimizer updates.

The training generator advances once for each sampled microbatch. Consequently, checkpoint only between complete global steps, after all n_{acc} microbatches and the optimizer update have finished.

5.3.2 Canonical execution order

For global step s , use the learning rate α_s . After the optimizer update, let `completed_steps = step + 1`. Decisions about validation, logging, and checkpointing use this completed-step count. If validation and checkpointing are both due at the same boundary, perform validation first, logging second, and checkpointing last. The final optimizer update must force all three actions even if it does not coincide with an ordinary interval.

The complete ordering is therefore

train mode $\rightarrow \alpha_s \rightarrow$ zero gradients $\rightarrow n_{\text{acc}}$ microbatches $\rightarrow$ backward accumulation $\rightarrow$ clip
 $\rightarrow$ optimizer update $\rightarrow$ validation $\rightarrow$ logging $\rightarrow$ checkpoint.

Treat `eval_interval`, `log_interval`, and `checkpoint_interval` as positive integers measured in *completed optimizer updates*. Also choose a positive `num_validation_batches`. Whenever validation is due, `evaluate_validation(...)` should sample exactly that many batches from the validation stream using `val_generator`, compute their cross-entropy losses under `model.eval()` and `torch.inference_mode()`, and return their arithmetic mean.

A canonical training path is:

```
for step in range(next_step, num_steps):
    model.train()
    lr = get_lr(step, ...)
    for group in optimizer.param_groups:
        group["lr"] = lr

    optimizer.zero_grad()
    train_loss = 0.0

    for _ in range(gradient_accumulation_steps):
        x, y = get_batch(
            train_tokens, ..., generator=train_generator
        )
        logits = model(x)
        microbatch_loss = cross_entropy(logits, y)
        (microbatch_loss / gradient_accumulation_steps).backward()
        train_loss += microbatch_loss.detach().item()

    train_loss /= gradient_accumulation_steps
    grad_norm = gradient_clipping(model.parameters(), max_norm)
    optimizer.step()

    completed_steps = step + 1
    final_step = completed_steps == num_steps
    should_validate = final_step or completed_steps % eval_interval == 0
    should_log = (
        should_validate or completed_steps % log_interval == 0
    )
    should_checkpoint = (
        final_step or completed_steps % checkpoint_interval == 0
    )

    if should_validate:
        validation_loss = evaluate_validation(...)

    if should_log:
        log_metrics(...)

    if should_checkpoint:
        save_checkpoint(..., next_step=completed_steps)
```

Validation samples must come from the validation stream and must be sampled with the independent `val_generator`. Save the model's current training/evaluation mode, call `model.eval()`, and perform the entire validation loop inside a `with torch.inference_mode():` context. Calling `model.eval()` selects evaluation behavior for modules whose forward pass differs between training and evaluation; it does not disable automatic differentiation. `torch.inference_mode()` disables gradient recording and additional autograd book-keeping, so validation does not construct computation graphs or retain intermediate values for a backward pass. This reduces memory use and generally makes evaluation faster. Average the losses from several independently sampled validation batches, then restore the previous mode with `model.train(was_training)`.

Before spending GPU time, repeatedly optimize one fixed minibatch; a correct implementation should drive its loss down sharply.

At minimum, log the completed-step count, mean unscaled training loss across the accumulated microbatches, validation loss when evaluated, learning rate, and pre-clipping gradient norm. When saving after global step `step`, store `next_step = step + 1` together with the model, optimizer, and both generator states. Because checkpointing follows validation, the saved validation-generator state includes any validation batches sampled at that boundary. On resume, begin from the restored `next_step`; the next training and validation samples will then continue from their saved generator states.

Problem: Put it together (5 report points)

Deliverable: Write a script that trains the Transformer LM on user-provided training and validation streams. At minimum, it should allow the user to configure model and optimizer hyperparameters, load both streams with `np.memmap`, periodically report training and validation loss, and save or resume a checkpoint from a user-provided path. Implement gradient accumulation and follow the execution and checkpoint ordering specified above.

Before beginning the full TinyStories run, use a small model configuration to confirm that one fixed minibatch can be overfit, that validation executes under `torch.inference_mode()`, and that the model's previous mode is restored after evaluation.

6 Generating Text

Now that we can train models, the last piece we need is the ability to generate text from our model. The model itself accepts batched inputs and returns logits of shape `(batch_size, sequence_length, vocab_size)`. For clarity, the decoding discussion below describes one sequence at a time, so a single sequence produces a matrix of logits of shape `(sequence_length, vocab_size)`. Each row is an unnormalized score vector for the token that follows that position; applying softmax converts it into a probability distribution.

6.1 Decoding

To generate text, provide the model with a non-empty sequence of prefix tokens, called the *prompt*, and use the final-position logits to obtain a distribution for the next token. Sampling a token, appending it, and repeating this procedure produces an autoregressive completion. A non-empty prompt is required because the decoder needs a final input position from which to obtain its first next-token logits.

Concretely, one step of the decoding process should take in a sequence $x_{1:i}$ and return a token x_{i+1} . Let

$$z_i = \text{TransformerLM}(x_{1:i})_i \in \mathbb{R}^{|\mathcal{V}|}$$

be the logits at the final input position. Then, for vocabulary item $c \in \mathcal{V}$,

$$p_\theta(x_{i+1} = c \mid x_{1:i}) = \frac{\exp(z_{i,c})}{\sum_{c' \in \mathcal{V}} \exp(z_{i,c'})}.$$

Here, `TransformerLM` is our model, which takes as input a sequence of length `sequence_length` and produces a matrix of size `(sequence_length, vocab_size)`, and we take the final row because it contains the next-token logits after position i .

This gives us a basic decoder by repeatedly sampling from these one-step conditionals until we generate `<|endoftext|>` or reach a user-specified maximum number of new tokens. Retain the complete prompt and completion for the returned output. The complete sequence may grow beyond `context_length`; at each decoding step, pass only its most recent `context_length` tokens to the model.

6.2 Decoder Tricks

We will be experimenting with small models, and small models can sometimes generate very low-quality texts. Two simple decoder tricks can help fix these issues. First, in *temperature scaling* we modify our softmax with a temperature parameter τ , where the new softmax is

$$\text{softmax}(z, \tau)_c = \frac{\exp(z_c/\tau)}{\sum_{c' \in \mathcal{V}} \exp(z_{c'}/\tau)}, \quad c \in \mathcal{V}.$$

Require $\tau > 0$. As $\tau \rightarrow 0^+$, the largest logit increasingly dominates and the softmax distribution approaches a one-hot vector concentrated on an argmax entry.

Second, another trick is *nucleus* or *top- p sampling*, where we modify the sampling distribution by truncating low-probability tokens. Let q be a probability distribution that we get from a temperature-scaled softmax of size `vocab_size`. Nucleus sampling with hyperparameter p produces the next token according to the equation

$$P(x_{i+1} = c \mid q) = \begin{cases} \frac{q_c}{\sum_{c' \in \mathcal{V}_p} q_{c'}}, & c \in \mathcal{V}_p, \\ 0, & \text{otherwise,} \end{cases}$$

where $0 < p \leq 1$ and $\mathcal{V}_p \subseteq \mathcal{V}$ contains the highest-probability vocabulary items and is the smallest such set satisfying $\sum_{c' \in \mathcal{V}_p} q_{c'} \geq p$. Compute this set by sorting the entries of q in descending order and selecting the shortest prefix whose cumulative probability reaches or exceeds p .

Problem: Decoding (5 report points)

Deliverable: Implement text generation with temperature scaling and top- p sampling. The following interface is recommended, but generation is not connected to an adapter or unit test; it is assessed through the outputs and analysis required in `REPORT.md`.

```
def generate(
    model: torch.nn.Module,
    prompt_ids: torch.Tensor,
    max_new_tokens: int,
    context_length: int,
    *,
    temperature: float = 1.0,
    top_p: float = 1.0,
    eot_token_id: int | None = None,
    generator: torch.Generator | None = None,
) -> torch.Tensor:
    ...
```

The input `prompt_ids` should be a non-empty, one-dimensional `torch.long` tensor on the model's device. Return a one-dimensional tensor containing the entire prompt followed by the generated completion. Require `max_new_tokens` ≥ 0 , `context_length` > 0 , `temperature` > 0 , and $0 < \text{top_p} \leq 1$. Require the model to expose the `context_length` attribute specified in [Section 3.5](#), and require the supplied `context_length` to equal `model.context_length`, so generation cannot silently crop to a different window than the model supports. If `max_new_tokens` is zero, return the prompt unchanged. When `eot_token_id` is not `None`, stop immediately after generating that token and retain it in the returned sequence.

Save the model's previous training/evaluation mode, call `model.eval()`, and run the decoding loop under `torch.inference_mode()`. At each step:

1. Crop the complete sequence to its most recent `context_length` tokens.
2. Add a batch dimension, run the model, and select the logits at the final sequence position.

3. Divide the logits by `temperature` and compute a numerically stable softmax.
4. Sort the probabilities together with their vocabulary indices, retain the shortest descending prefix whose cumulative probability reaches `top_p`, zero the remaining probabilities, and renormalize.
5. Sample a rank from this filtered sorted distribution with `torch.multinomial`, passing `generator` when it is supplied, then map that sampled rank back through the saved sorted vocabulary indices to obtain the actual token ID. Equivalently, scatter the filtered probabilities back to the original vocabulary order before sampling. Append the resulting token ID to the complete sequence.

Restore the model’s previous mode before returning. Because this assignment does not implement a KV cache, each iteration recomputes the model activations for the entire cropped context. Demonstrate and analyze this decoder in `REPORT.md` as specified in [Section 7.3](#).

7 Training your modern Transformer LM on TinyStories

The final experiment holds the model architecture and number of optimizer updates fixed. Your task is to use controlled short runs to tune the training procedure, select a defensible configuration, and train that configuration for the complete budget. The goal is not merely to produce the smallest loss you happen to observe; it is to record enough evidence to explain which choices helped, which did not, and how the trained model behaves under different decoding strategies.

7.1 Fixed budget and recommended starting point

The architecture in [Section 3.1](#), including `context_length = 256`, is fixed. Training sequences also have fixed length `sequence_length = 256`. Every final run performs exactly

$$N_{\text{updates}} = 10,000$$

optimizer updates, indexed by $s = 0, \dots, 9999$. Consequently, use $s_c = 9999$ as the cosine endpoint: the first update uses α_0 and the final update uses $\alpha_{9999} = \alpha_{\min}$. The budget is expressed in optimizer updates rather than wall-clock time because runtime depends on hardware and implementation details.

The values below are a recommended starting point. Apart from the fixed sequence length, update count, and cosine endpoint, these are training hyperparameters that you may change during controlled tuning.

Symbol	Variable	Value	Meaning and notes
n	<code>sequence_length</code>	256	tokens per sequence; fixed
B	<code>batch_size</code>	16	sequences per microbatch
n_{acc}	<code>gradient_accumulation_steps</code>	16	microbatches per optimizer update
$N_{\text{tok/update}}$	<code>tokens_per_step</code>	65,536	derived as Bnn_{acc} at the starting point
N_{updates}	<code>num_steps</code>	10,000	optimizer updates; fixed
$\alpha_{\max}$	<code>learning_rate_max</code>	3×10^{-4}	peak learning rate
$\alpha_{\min}$	<code>learning_rate_min</code>	3×10^{-5}	final learning rate
s_w	<code>warmup_steps</code>	200	warmup endpoint; the peak is first reached at step $s = s_w$
s_c	<code>cosine_steps</code>	9,999	final cosine-decay step; fixed
(β_1, β_2)	<code>betas</code>	(0.9, 0.95)	AdamW moment coefficients
$\varepsilon_{\text{Adam}}$	<code>adam_eps</code>	10^{-8}	AdamW numerical-stability constant
λ	<code>weight_decay</code>	0.1	decoupled weight decay
M	<code>max_grad_norm</code>	1.0	global gradient-clipping threshold

At this starting point, every optimizer update uses $16 \times 256 \times 16 = 65,536$ sampled tokens, and the complete run uses 655,360,000 sampled tokens. You may vary B and n_{acc} , including reducing the microbatch size

to fit memory. The authoritative budget remains 10,000 optimizer updates, so changing the effective batch changes the number of sampled tokens used by the run. Record B , n_{acc} , Bn_{acc} , tokens per optimizer update, and total sampled tokens in your report.

7.2 Controlled short-run tuning

Hyperparameter comparisons are useful only when the comparison itself is controlled. Begin with the recommended configuration and choose a short cap, normally between 1,000 and 2,000 optimizer updates. Use the same cap, model-initialization seed, training-generator seed, validation-generator seed, evaluation frequency, and validation procedure for every run in that comparison. Keep $s_c = 9999$ rather than compressing the cosine schedule to the short-run cap; a short run should observe the same prefix of the learning-rate schedule that it would see during the full run.

Construct an *ablation ladder*: change one training hyperparameter from the current baseline, compare the resulting validation-loss trajectory and stability, and retain the better-supported choice as the baseline for the next comparison. Useful quantities to inspect include training and validation loss, learning rate, pre-clipping gradient norm, and signs of divergence or stalled learning. At minimum, compare one baseline with one one-factor variant. You may run as many additional comparisons as your compute budget permits.

If you compare effective batch sizes, remember that the fixed optimizer-step cap then exposes the models to different numbers of sampled tokens. Report this explicitly instead of treating the runs as equal-token comparisons. After the short-run ladder, select one configuration and explain why it deserves the full 10,000-update budget.

7.3 Final TinyStories run and evaluation

Download the data as described in [Section 2.4](#) and train using the command-line interface designed in [Section 5.3](#). Use the checkpoint procedure from [Section 5.2](#).

After the 10,000 updates, measure the result with a standardized final evaluation. Create a fresh validation generator with seed 42 and average the cross-entropy across 100 independently sampled validation batches, each containing 16 sequences of length 256. This fresh generator makes the reported evaluation independent of how frequently validation ran during training. Report

$$\text{perplexity} = \exp(\text{mean validation cross-entropy}).$$

Do not average per-batch perplexities. First average the cross-entropy losses and exponentiate that single mean.

There is no exact hardware-dependent loss target and no leaderboard. Training quality is assessed from the completeness and quality of the experimental evidence in `REPORT.md`, not from matching one reference number.

Problem: Train, evaluate, and analyze (20 report points)

Deliverable: Complete `REPORT.md` and submit it together with the training logs. The report must contain:

- the exact commands, random seeds, and final training configuration;
- a table of all attempted configurations, including the short-run cap and the one controlled change in each comparison;
- an explanation of the evidence used to choose the full-run configuration;
- microbatch size, accumulation count, effective batch size, tokens per optimizer update, and total sampled tokens;
- training- and validation-loss curves against completed optimizer updates, together with the

learning-rate curve;

- final validation loss and perplexity from the standardized 100-batch evaluation;
- a discussion of convergence, instability, overfitting, stalled learning, or diminishing returns observed during the experiments;
- complete output from `uv run pytest` and the final training logs;
- the SHA-256 digest of the submitted `final_model.pt`;
- the decoding experiment described below.

For the decoding experiment, choose one non-empty prompt and hold that prompt, the random seed, and the maximum completion length fixed throughout. Compare at least three temperature values while holding $\text{top-}p$ fixed, then compare at least three $\text{top-}p$ values while holding temperature fixed. You choose and justify the numerical settings. Include the generated outputs and discuss coherence, diversity, repetition, prompt adherence, degeneration, and EOT behavior. Generation has no adapter or unit test; its implementation and behavior are graded through this evidence and analysis.

Submit your implementation through the assignment repository according to the course submission instructions. The report must link to the figures under `report_assets/` and to the required machine-readable evidence under `logs/`. The model, metrics, logs, and generations must all come from the same final run. Do not commit the dataset, `final_model.pt`, or any full training checkpoint to Git.

7.4 Final model artifact and LMS submission

The LMS submission includes one model-only artifact named `final_model.pt`. It must be a plain state dictionary corresponding to the model used for the standardized validation and decoding study. Move every state-dictionary tensor to CPU and convert floating-point tensors to `torch.float16`; preserve the dtype of non-floating tensors. For example, the export can follow this pattern:

```
state = {
    name: tensor.detach().cpu().to(torch.float16)
    if tensor.is_floating_point()
    else tensor.detach().cpu()
    for name, tensor in model.state_dict().items()
}
torch.save(state, "final_model.pt")
```

This artifact contains only the model state. Do not include optimizer state, RNG state, hyperparameters, or other checkpoint fields in `final_model.pt`. Continue to use the complete checkpoint format from [Section 5.2](#) for resuming training, but do not submit those larger checkpoints. The fixed architecture contains 19,272,192 parameters, so the FP16 model artifact is approximately 39 MB.

The required evidence files are:

- `logs/final_training.csv`, with columns `completed_steps`, `train_loss`, `validation_loss`, `learning_rate`, and `grad_norm`;
- `logs/final_metrics.json`, recording the completed update count, standardized validation seed, number of validation batches, validation batch size, sequence length, mean validation loss, and perplexity under the keys `completed_steps`, `validation_seed`, `validation_batches`, `validation_batch_size`, `sequence_length`, `validation_loss`, and `perplexity`;
- `logs/generations.json`, recording for every reported sample the prompt text and token IDs, output text and token IDs, temperature, $\text{top-}p$, sampling seed, and maximum number of new tokens under the keys `prompt_text`, `prompt_ids`, `output_text`, `output_ids`, `temperature`, `top_p`, `seed`, and `max_new_tokens`. Store the records as a JSON list with at least six entries; and
- every figure embedded by `REPORT.md`, stored under `report_assets/`.

From the repository root, run `bash make_submission.sh`. The script runs the public tests, records their output, validates the required artifacts, and creates `submission.zip`. Test failures do not prevent packaging, so partial work can still be submitted, but they are recorded in the archive. The archive contains:

```
submission.zip
|-- src/
|-- tests/adapters.py
|-- REPORT.md
|-- report_assets/
|-- logs/
|   |-- final_training.csv
|   |-- final_metrics.json
|   |-- generations.json
|-- final_model.pt
|-- public_tests.txt
`-- submission_manifest.json
```

The generated manifest records the Git revision and working-tree status, the public-test exit status, and SHA-256 digests for the submitted model, report, and evidence files. Graders may load the submitted state dictionary through `adapters.get_transformer_lm`, repeat the standardized validation, and run additional prompts. A difference greater than 0.02 nats/token between the reported and recomputed standardized validation loss is flagged for manual review rather than automatically scored as a wrong answer. This consistency check connects the report to the submitted code and model; it does not replace the course’s ordinary academic-integrity procedures.

7.5 Why 19 million parameters?

With untied embeddings, the full model has

$$2|\mathcal{V}|d_{\text{model}} + L \left[2d_{\text{model}}^2 + 2d_{\text{model}}h_{kv}d_h + 3d_{\text{model}}d_{ff} + 2d_{\text{model}} \right] + d_{\text{model}} = 19,272,192$$

trainable parameters.

The Chinchilla scaling-law study [J. Hoffmann et al., 2022] found that, under a fixed training-compute budget, model size and the number of training tokens should grow at approximately the same rate. Its compute-optimal result is commonly summarized as roughly 20 training tokens per model parameter. Applying that heuristic to 19,272,192 parameters gives about 385 million tokens. Our complete TinyStories training stream contains 466,876,982 tokens, or approximately 24.2 corpus tokens per parameter, so the amount of distinct course data is in the same broad scaling regime.

At the recommended effective batch, the 10,000-update run samples 655,360,000 tokens, or approximately 34.0 sampled tokens per parameter and 1.40 corpus-equivalents. These are not literal passes or epochs: random windows are sampled with replacement, so some regions may be sampled repeatedly while others may not be sampled during a particular run.

This calculation is a design heuristic rather than a promised optimum. The Chinchilla fits were obtained for much larger models trained on broader text distributions, whereas TinyStories is deliberately narrow and synthetically generated. It nevertheless explains the pairing used in this assignment: the model is large enough to make meaningful use of the corpus, while the dataset supplies enough distinct next-token observations to avoid a parameter count that is obviously too large for the available data.

Bibliography

- [1] R. Eldan and Y. Li, “TinyStories: How Small Can Language Models Be and Still Speak Coherent English?” 2023.

- [2] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proc. of ACL*, 2016.
- [3] C. Wang, K. Cho, and J. Gu, “Neural Machine Translation with Byte-Level Subwords.” 2019.
- [4] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [5] A. Vaswani et al., “Attention is All you Need,” in *Proc. of NeurIPS*, 2017.
- [6] T. Q. Nguyen and J. Salazar, “Transformers without Tears: Improving the Normalization of Self-Attention,” in *Proc. of IWSLT*, 2019.
- [7] R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” in *Proc. of ICML*, 2020.
- [8] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization.” 2016.
- [9] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models.” 2023.
- [10] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” in *Proc. of NeurIPS*, 2019.
- [11] A. Grattafiori et al., “The Llama 3 Herd of Models.” [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [12] A. Yang et al., “Qwen2.5 Technical Report,” arXiv preprint arXiv:2412.15115, 2024.
- [13] A. Chowdhery et al., “PaLM: Scaling Language Modeling with Pathways.” 2022.
- [14] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning.” [Online]. Available: <https://arxiv.org/abs/1702.03118>
- [15] P. Ramachandran, B. Zoph, and Q. V. Le, “Searching for Activation Functions.” [Online]. Available: <https://arxiv.org/abs/1710.05941>
- [16] Y. N. Dauphin, A. Fan, M. Auli, and D. Grangier, “Language Modeling with Gated Convolutional Networks.” [Online]. Available: <https://arxiv.org/abs/1612.08083>
- [17] N. Shazeer, “GLU Variants Improve Transformer.” 2020.
- [18] J. Su, Y. Lu, S. Pan, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding.” 2021.
- [19] S. Black et al., “GPT-NeoX-20B: An Open-Source Autoregressive Language Model.” [Online]. Available: <https://arxiv.org/abs/2204.06745>
- [20] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.” 2023.
- [21] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *Proc. of ICLR*, 2015.
- [22] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. of ICLR*, 2019.
- [23] T. B. Brown et al., “Language Models are Few-Shot Learners,” in *Proc. of NeurIPS*, 2020.
- [24] J. Hoffmann et al., “Training Compute-Optimal Large Language Models.” 2022.
- [25] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” in *Proc. of ICLR*, 2020.